AI Engineer Assignment

Ship a change to a live game.

TIME	TOOLS	YOU'LL SEND
About 4 hours	**AI, docs, anything**	**Written answer + AI logs**

01 Why this assignment

Our games are live, with real players and real money. Every change ships into a system that already exists.

AI can write a design in seconds. We want to see the judgment behind the design: what you noticed in the facts, what you chose, and why.

> *Chaos Engine Labs is a Bengaluru-based, AI-native studio building casual mobile games. We're backed by marquee investors, live in production with real players and real revenue, and built by a founding team that has run growth, monetisation and content at scale across gaming and consumer internet.*

02 What we're looking for

Self-teaching & AI fluency	You use AI as a real collaborator. You give it the facts, you check what it says, and you correct it when it is wrong.
Reading the system	You read the facts closely and point to the exact evidence for what you say.
Systems thinking	Your design works as a whole, not only on paper.
Judgment	You make clear trade-offs, you say who wins and who loses, and you know which decisions belong to someone else.

03 The challenge

The game: **Merge Kingdom**

Merge Kingdom is a mobile puzzle game. The game saves player progress on the phone. It also uploads the save to our server, so that players can play on more than one device.

The change: **two currencies in version 2.5**

Today the game has one currency: coins. Players earn coins by playing, and they can also buy coin packs with real money. Finance needs paid currency kept separate. So in version 2.5:

- **coins** are earned by playing only.
- **gems** are bought with real money only. The store will sell items for gems.

> **The rule:** no player may lose progress, or anything that they paid for.

Fact 1: **current save data (example)**

```
{
  "player_id": "p_8812",
  "saved_at": 1761700000,
  "level": 142,
  "coins": 5400,
  "inventory": { "hammer": 3, "shuffle": 1 },
  "no_ads": true
}
```

`saved_at` is the time of the save, in seconds, from the phone clock.

Fact 2: **save code in the app** (versions 2.2, 2.3 and 2.4 have the same code)

```
[Serializable]
public class Inventory {
    public int hammer;
    public int shuffle;
}

[Serializable]
public class SaveData {
    public string player_id;
    public long saved_at;
    public int level;
    public int coins;
    public Inventory inventory;
    public bool no_ads;
}

public static class SaveSystem {
    public static SaveData Current;

    // Called when the game starts.
    public static void Load() {
        string json = File.ReadAllText(LocalPath);
        Current = JsonUtility.FromJson<SaveData>(json);
    }

    // Called after every level and every purchase.
    public static void Save() {
        Current.saved_at = DateTimeOffset.Now.ToUnixTimeSeconds();
        string json = JsonUtility.ToJson(Current);
        File.WriteAllText(LocalPath, json);
        CloudSync.Upload(json);
    }

    // Called when the server sends a save to this device.
    public static void OnCloudSave(string serverJson) {
        SaveData server = JsonUtility.FromJson<SaveData>(serverJson);
        if (server.saved_at > Current.saved_at) {
            Current = server;
            File.WriteAllText(LocalPath, serverJson);
        }
    }
}
```

Fact 3: **server sync rule**

- The server keeps one save for each player. When an upload arrives, the server keeps it only if its `saved_at` is larger than the stored `saved_at`. The server does not read the other fields.
- If the server keeps the upload, it sends it to the other devices of the player. If the server does not keep it, it sends its stored save back to the device that uploaded. On the device, both arrive through `OnCloudSave`.
- When a device is offline, `CloudSync.Upload` keeps the upload in a queue and sends it when the device is online again.

Fact 4: **purchase records**

The server has a record of each coin pack that a player bought: player, pack, number of coins, time. The server has no record of how players spent coins. Coins are spent on the phone.

Fact 5: **players by app version** (daily players, last 7 days)

APP VERSION	SHARE OF DAILY PLAYERS	NOTES
2.4 (latest)	58%	
2.3	21%	
2.2	11%	
2.0 – 2.1	10%	About half are on Android 7. New builds do not support Android 7, so they cannot update.

- About 9% of players used more than one device in the last 30 days.
- A new build takes 3 to 7 days for store review and rollout.
- Our last forced update lost 18% of the players who saw the update screen.

Fact 6: **messages from players to support** (last month)

1. "I played on my tablet all weekend and got to level 160. My son played on my old phone in airplane mode on Sunday night, after I stopped. Today I am back at level 151. Where did my levels go?"
2. "I changed the date on my phone to tomorrow, opened the free chest early and played one level. Then I changed the date back. Later I played more levels on the bus with no internet. When I got home, those levels were gone."
3. "I paid for No Ads in March. Last week the ads came back. I play on my phone and on my old tablet."

Your task: **do all six**

Use only the facts above. If you need a fact that we did not give, write your assumption and continue.

1. **Read the current system.** Describe what in the current save and sync system matters for this change. For each point, give the exact code line, table row or message that shows it.
2. **Explain the support messages.** For each of the three messages, explain step by step what happened on each device and on the server.
3. **Design version 2.5.** Describe the new save data, what the app does and what the server does. Your design must keep the rule.
4. **Split the existing coins.** Choose a rule to split each player's current coins into coins and gems. Show your rule on two example players. Say who gains, who loses, and what it

costs the company.

5. **Plan the release.** Give the order of the steps. Say what you check before each step, how you will know if something goes wrong, and how you undo it.
6. **Find the limits.** Say where your design can still fail. List the questions that you would ask the product manager or Finance before you build it.

04 What to send us

1. **Your written answer** to the six tasks, as one document. Plain language is fine.
2. **Your full AI logs:** share links or exports of every AI conversation that you used. Send them as they are. Do not clean them up or summarise them. Keep your AI conversation. You will use it in the next round.
3. **A few lines on how you used AI:** what you asked, what you kept, what you rejected, and where the AI was wrong.
4. **Time spent,** roughly.

05 How to approach it

- **Evidence beats volume.** Point to the line, the row or the message. A short answer that is right beats a long one that is general.
- **Use AI, and show us how.** We expect you to use AI. Your AI logs are part of what we evaluate.
- **There is no single correct design.** We read your reasoning and how you chose between the options.
- **State your assumptions.**
- **Send it within 48 hours** of the start date that you give us. The work itself should take about 4 hours. A short, correct answer is better than a long one.

---

06 What happens next

Once you submit, we will review your answers.

To get started, reply with your planned start date and submission date. If anything is unclear, use your judgment, make your own interpretation and tell us what you assumed.

We look forward to seeing how you think.